



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

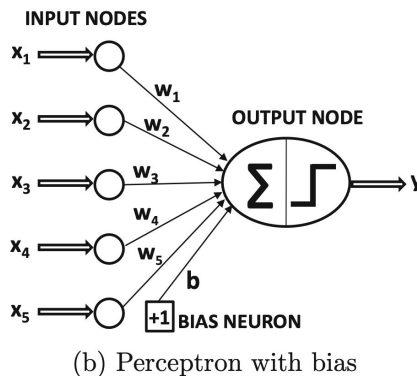
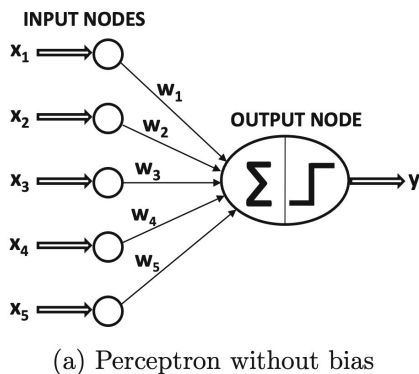
Tecnología Digital VI: Inteligencia Artificial

Backpropagation + Taller MNIST

Perceptrón simple

Una **red neuronal** toma un vector de **entrada** de p variables $X = (X_1, X_2, \dots, X_p)$ y construye una **función** no-lineal $f(X)$ para predecir la **respuesta** Y .

Un **perceptrón simple** representa la red neuronal más simple posible: contiene **una sola neurona**. De hecho, no califica como profundo.



Algunas aclaraciones:

- En la **capa de entrada** en sí misma no se lleva a cabo **ningún cálculo**.
- Después de la **sumatoria**, que junto a los **w** , representa la **combinación lineal**, se aplica la **función de activación**, que en la figura en cuestión se optó por la **función signo**.
- El **sesgo** refiere a la parte invariante de la predicción y es incorporado como un peso a un eje, usando una **neurona de sesgo**, que siempre transmite el valor **1**.

Perceptrón simple | Aplicación

Un banco desea implementar un sistema de **reconocimiento automático de cheques**.

MR. JOHN JONES
1645 DUNDAS ST. W, APT. 27
TORONTO, ON M6K 1V2

DATE 20061201
Y Y Y Y M M D D

PAY TO THE ORDER OF Wikimedia Foundation \$ 100.55

One Hundred Dollars and 55/100 DOLLARS

 FIRST BANK OF WIKI
Victoria Main Branch
1425 James St., P.O. Box 4001
Victoria (B.C.) V8X 3X4

MEMO Donation

John Jones MP

⑈ 243 ⑈ ⑆ 00005 ⑆ 23 ⑆ 123 ⑆ 456 ⑆ 7 ⑈

Perceptrón monocapa

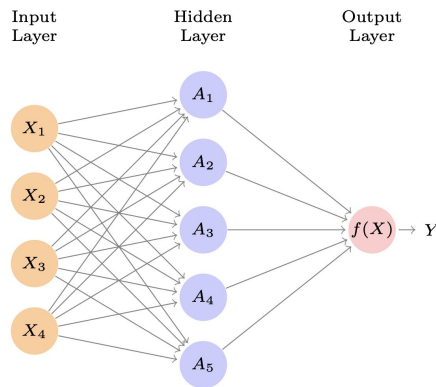


FIGURE 10.1. Neural network with a single hidden layer. The hidden layer computes activations $A_k = h_k(X)$ that are nonlinear transformations of linear combinations of the inputs $X_1, X_2, \dots, X_p$. Hence these A_k are not directly observed. The functions $h_k(\cdot)$ are not fixed in advance, but are learned during the training of the network. The output layer is a linear model that uses these activations A_k as inputs, resulting in a function $f(X)$.

Los p **predictores** conforman las unidades de la **capa de entrada**. Cada una de esas unidades alimenta a las K unidades de la **capa oculta**. K es un hiperparámetro; en la figura, $K = 5$.

Cada A_k es una transformación diferente de los atributos originales. Concretamente, resulta de **combinar linealmente** los atributos originales y, **luego**, a esta combinación aplicarle una función g , conocida como **función de activación**. Es esencial que esta función g sea **no-lineal** para evitar que el modelo colapse en uno lineal simple y para permitir al modelo capturar no-linealidades complejas y efectos de interacción. Los nuevos atributos resultantes son luego alimentados a la(s) unidad(e)s de la **capa de salida**.

Perceptrón multicapa

Las redes neuronales modernas suelen tener **más de una capa oculta** y cada una de ellas con varias neuronas.

Si bien, en teoría, un perceptrón monocapa con un gran número de neuronas en su única capa tiene la habilidad de aproximar casi todas las funciones, el proceso de **aprendizaje** para encontrar una buena solución se torna **mucho más sencillo** si, en cambio, usamos **varias capas, cada una de ellas con una menor cantidad de neuronas**.

La **primera capa oculta** funciona de la misma manera que en el perceptrón monocapa.

Las **siguientes capas ocultas** tratan a las activaciones de la capa oculta anterior como entradas y computan nuevas activaciones.

Perceptrón multicapa

A la arquitectura específica de un perceptrón multicapa (también conocida como red neuronal multicapa) se la conoce como red **feed-forward** ($\approx$ alimentación hacia adelante), porque capas sucesivas se alimentan entre sí en la dirección de avance desde la entrada hacia la salida.

La arquitectura por defecto de las redes feed-forward asume que todas las neuronas en una capa están conectadas a las neuronas de la capa siguiente. A dichas capas se las conoce como capas **fully-connected** ($\approx$ completamente conectadas).

En la figura de la derecha, podemos ver **distintas formas de representar** a un mismo perceptrón multicapa.

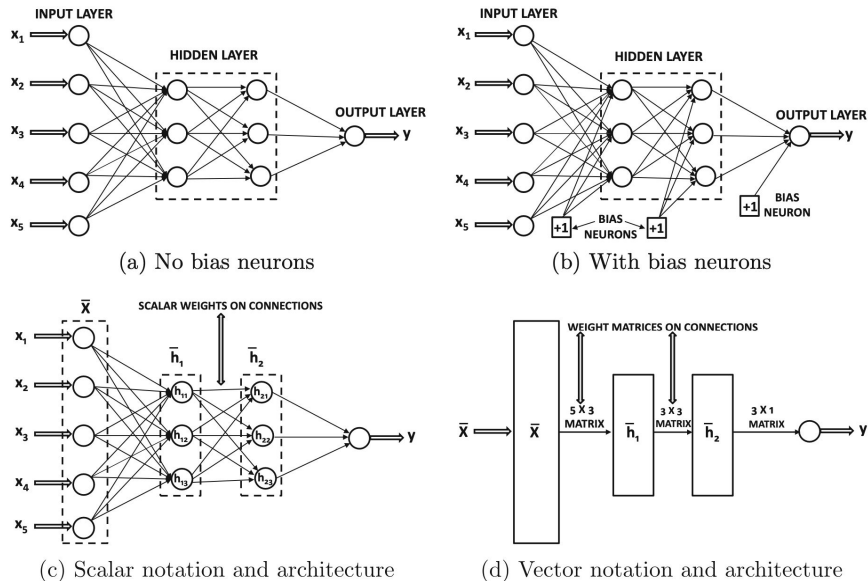


Figure 1.11: The basic architecture of a feed-forward network with two hidden layers and a single output layer. Even though each unit contains a single scalar variable, one often represents all units within a single layer as a single vector unit. Vector units are often represented as rectangles and have connection *matrices* between them.

Perceptrón multicapa

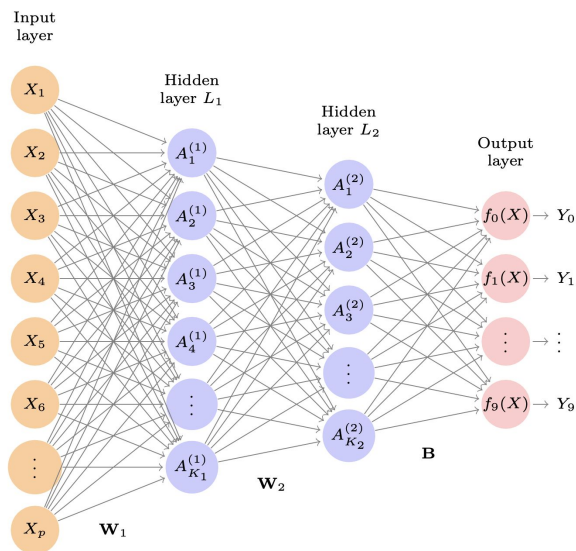


FIGURE 10.4. Neural network diagram with two hidden layers and multiple outputs, suitable for the MNIST handwritten-digit problem. The input layer has $p = 784$ units, the two hidden layers $K_1 = 256$ and $K_2 = 128$ units respectively, and the output layer 10 units. Along with intercepts (referred to as biases in the deep-learning community) this network has 235,146 parameters (referred to as weights).

Perceptrón multicapa | Derivada de la Loss

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial z^{[3]}} \frac{\partial z^{[3]}}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z^{[2]}} \frac{\partial z^{[2]}}{\partial W^{[2]}}$$



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Backpropagation + AutoGrad

AutoGrad

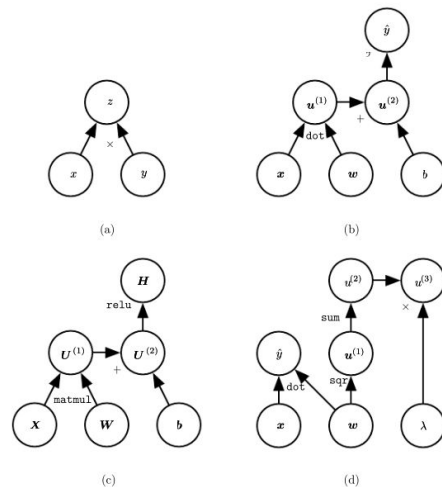
Las librerías de Redes Neuronales están preparadas para derivar cualquier cosa.

Se aprovechan de la **Regla de la Cadena** para partir las fórmulas en pequeñas componentes fundamentales (**Diferenciación Automática**).

En Pytorch este módulo se llama **AutoGrad**.

AutoGrad parte todas las operaciones necesarias en un **grafo computacional**

<https://colab.research.google.com/drive/1NLRgRI71DPLL>
=sharing



Backpropagation

- Se hace una pasada **forward**, guardando los valores de activación de las neuronas.
- Se calculan las **derivadas** retroactivamente respecto a los pesos.
- Se modifican los pesos en base a los valores de las **derivadas** multiplicadas por un learning rate (**Gradient Descent**).
- Hay una forma “sistemática” de hacer **backpropagation**

Taller MNIST

Perceptrón simple | MNIST

Las imágenes corresponden, concretamente, al popular conjunto de datos conocido como **MNIST** (*modified National Institute of Standards and Technology*).

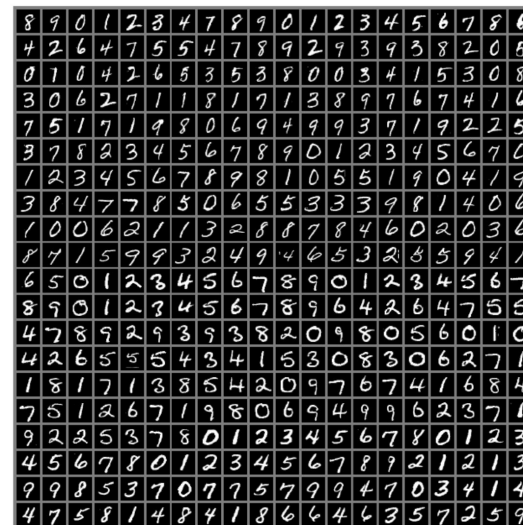


Figure 1.9: Example inputs from the MNIST dataset. The “NIST” stands for National Institute of Standards and Technology, the agency that originally collected this data. The “M” stands for “modified,” since the data has been preprocessed for easier use with machine learning algorithms. The MNIST dataset consists of scans of handwritten digits and associated labels describing which digit 0-9 is contained in each image. This simple classification problem is one of the simplest and most widely used tests in deep learning research. It remains popular despite being quite easy for modern techniques to solve. Geoffrey Hinton has described it as “the *drosophila* of machine learning,” meaning that it enables machine learning researchers to study their algorithms in controlled laboratory conditions, much as biologists often study fruit flies.

Perceptrón simple | MNIST

La idea es construir un modelo para **clasificar imágenes** en la clase correcta de **dígito 0-9**.

Hay **60.000** imágenes de **entrenamiento** y **10.000** de **evaluación**.

Cada imagen tiene $p = 28 * 28 =$ **784** **píxeles**.

Cada **pixel** es un valor en una **escala de grises de 8 bits**, entre 0 y 255, representando la cantidad relativa del dígito escrito en ese pequeño cuadrado.

Estos píxeles son almacenados en el **vector de entrada X** .

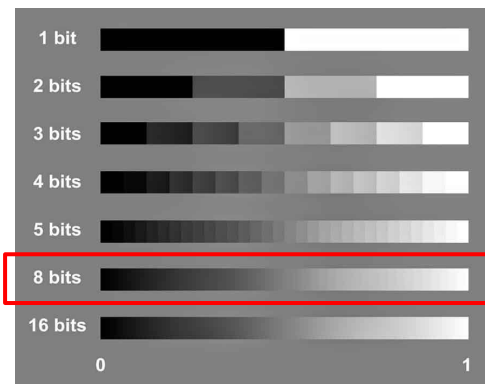


Image Size	Possible combinations (2^k)
1-bit	2
2-bit	4
4-bit	16
8-bit	256
16-bit	65536

8 bits = 256 combinaciones posibles de ceros y unos



0 0 1 1 0 1 0 0

Perceptrón simple | MNIST

La **salida** es un **vector de 10 variables dicotómicas**, con un 1 en la clase correspondiente y 0s para el resto.

Utilizaremos la **función de activación softmax**, que asegura que las entradas que reciba se transformen en **probabilidades**; es decir, en **valores no-negativos y que suman 1**. (No usamos la función sigmoide porque, si bien genera valores entre 0 y 1, no garantiza que sumen 1).

El modelo estimará, para cada observación, la probabilidad para cada una de las 10 clases y, luego, a cada observación se le asignará **la clase con la mayor probabilidad**. (Si hubiese empate entre dos o más clases posibles, se selecciona una de ellas de forma aleatoria).

